

Strutture dati

Dizionari

Come associare informazioni tramite chiavi e valori, come un vero vocabolario.

Cosa sono i dizionari?

Un **dizionario** è una raccolta di informazioni organizzate in coppie **chiave → valore**.

Funziona come un vero vocabolario: cerchi una parola (la **chiave**) e trovi la sua definizione (il **valore**). Oppure come una rubrica telefonica: cerchi il nome di una persona e trovi il suo numero.

```
"nome" → "Alice"  
"eta" → 16  
"citta" → "Roma"
```

Creare un dizionario

I dizionari si scrivono con le **parentesi graffe** `{}`, con coppie **"chiave": valore** separate da virgole:

```
studente = {  
    "nome": "Alice",  
    "eta": 16,  
    "classe": "3A"  
}  
  
vuoto = {} # dizionario senza elementi
```

Accedere a un valore

Usi il nome della chiave tra parentesi quadre, come se fosse un indice:

```
studente = {"nome": "Alice", "eta": 16, "classe": "3A"}

print(studente["nome"])    # Alice
print(studente["eta"])     # 16
```

Se la chiave non esiste, ottieni un errore. Per evitarlo, usa `.get()`, che restituisce `None` (o un valore di default) invece di causare errore:

```
print(studente.get("cognome"))    # None – nessun errore
print(studente.get("cognome", "N/D")) # N/D – valore di default
```

Aggiungere e modificare

I dizionari sono modificabili. Assegna un valore a una chiave: se la chiave esiste, modifica il valore; se non esiste, crea una nuova coppia:

```
studente = {"nome": "Alice", "eta": 16}

studente["eta"] = 17          # modifica il valore esistente
studente["cognome"] = "Rossi" # aggiunge una nuova coppia

print(studente)
# {'nome': 'Alice', 'eta': 17, 'cognome': 'Rossi'}
```

Rimuovere elementi

```
studente = {"nome": "Alice", "eta": 16, "classe": "3A"}

del studente["classe"]      # elimina la coppia "classe"
print(studente)             # {'nome': 'Alice', 'eta': 16}

eta = studente.pop("eta")    # rimuove e restituisce il valore
print(eta)                  # 16
print(studente)             # {'nome': 'Alice'}
```

Strumenti utili

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Tutte le chiavi
print(persona.keys())       # dict_keys(['nome', 'eta', 'citta'])

# Tutti i valori
print(persona.values())     # dict_values(['Alice', 16, 'Roma'])

# Tutte le coppie chiave-valore
print(persona.items())      # dict_items([('nome', 'Alice'), ('eta', 16), ('citta', 'Roma')])

# Verificare se una chiave esiste
print("nome" in persona)    # True
print("cognome" in persona) # False
```

Scorrere un dizionario

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Solo le chiavi
for chiave in persona:
    print(chiave)    # nome, eta, citta

# Chiavi e valori insieme
for chiave, valore in persona.items():
    print(f"{chiave}: {valore}")
```

Output:

```
nome: Alice
eta: 16
citta: Roma
```

Dizionari dentro dizionari

I valori possono essere qualsiasi cosa, anche altri dizionari. Utile per rappresentare strutture complesse:

```
classe = {
    "Alice": {"eta": 16, "voto": 8},
    "Bob":   {"eta": 17, "voto": 7}
}

print(classe["Alice"]["voto"]) # 8
print(classe["Bob"]["eta"])    # 17
```

Liste

Come raccogliere più valori in una sequenza ordinata e modificabile.

Cosa sono le liste?

Immagina un foglio con una lista della spesa: ci scrivi sopra gli elementi uno sotto l'altro, puoi aggiungerne di nuovi, cancellarne, cambiarli. In Python le **liste** funzionano esattamente così.

Una lista è una raccolta ordinata di elementi. Puoi metterci qualsiasi tipo di valore, anche misti, e puoi modificarla liberamente dopo averla creata.

Creare una lista

Le liste si scrivono con le **parentesi quadre** `[]`, con gli elementi separati da virgole:

```
frutta = ["mela", "banana", "ciliegia"]
numeri = [1, 2, 3, 4, 5]
mista = [1, "ciao", True, 3.14]    # anche tipi diversi vanno bene
vuota = []                        # lista senza elementi
```

Accedere agli elementi

Ogni elemento ha una **posizione** (chiamata indice) che parte da `0`, non da `1`. È una convenzione tipica della programmazione.

```
frutta = ["mela", "banana", "ciliegia"]
print(frutta[0])    # mela      - il primo elemento
print(frutta[1])    # banana    - il secondo
print(frutta[-1])   # ciliegia  - l'ultimo (conta dalla fine)
print(frutta[-2])   # banana    - il penultimo
```

Estrarre una parte della lista (slicing)

Puoi estrarre una "fetta" della lista indicando da dove a dove:

```
numeri = [0, 1, 2, 3, 4, 5]
print(numeri[1:4])    # [1, 2, 3]    – dalla posizione 1 alla 3 (4 esclusa)
print(numeri[:3])     # [0, 1, 2]    – dall'inizio alla posizione 2
print(numeri[3:])     # [3, 4, 5]    – dalla posizione 3 alla fine
print(numeri[::-1])   # [5, 4, 3, 2, 1, 0] – la lista al contrario
```

Modificare gli elementi

A differenza delle tuple (che vedremo dopo), le liste **si possono modificare**:

```
frutta = ["mela", "banana", "ciliegia"]
frutta[1] = "kiwi"    # sostituisce "banana" con "kiwi"
print(frutta)         # ['mela', 'kiwi', 'ciliegia']
```

Aggiungere elementi

Python ti dà tre modi per aggiungere elementi a una lista, a seconda di dove vuoi inserirli:

```
frutta = ["mela", "banana"]

frutta.append("ciliegia")    # aggiunge alla fine
print(frutta) # ['mela', 'banana', 'ciliegia']

frutta.insert(1, "kiwi")     # inserisce nella posizione 1
print(frutta) # ['mela', 'kiwi', 'banana', 'ciliegia']

altra = ["uva", "pesca"]
frutta.extend(altra)         # aggiunge tutti gli elementi di un'altra
                              # lista
print(frutta) # ['mela', 'kiwi', 'banana', 'ciliegia', 'uva', 'pesca']
```

Rimuovere elementi

Anche per rimuovere hai più opzioni, a seconda di cosa conosci: il valore o la posizione:

```
frutta = ["mela", "banana", "ciliegia", "banana"]

frutta.remove("banana") # rimuove la prima "banana" che trova
print(frutta) # ['mela', 'ciliegia', 'banana']

frutta.pop() # rimuove e restituisce l'ultimo elemento
print(frutta) # ['mela', 'ciliegia']

frutta = ["mela", "banana", "ciliegia"]
del frutta[1] # elimina l'elemento in posizione 1
print(frutta) # ['mela', 'ciliegia']

frutta.clear() # svuota completamente la lista
print(frutta) # []
```

Cercare e contare

```
numeri = [1, 2, 3, 2, 4, 2]
print(numeri.count(2)) # 3 - quante volte compare il numero 2
print(numeri.index(3)) # 2 - in quale posizione si trova il numero 3
print(len(numeri)) # 6 - quanti elementi ha la lista
```

Ordinare una lista

```
numeri = [3, 1, 4, 1, 5, 9, 2]

numeri.sort()          # ordina la lista (modifica l'originale)
print(numeri) # [1, 1, 2, 3, 4, 5, 9]

numeri.sort(reverse=True) # ordine decrescente
print(numeri) # [9, 5, 4, 3, 2, 1, 1]

# sorted() non modifica l'originale, restituisce una nuova lista
originale = [3, 1, 4, 1, 5]
ordinata = sorted(originale)
print(originale) # [3, 1, 4, 1, 5] - invariata
print(ordinata)  # [1, 1, 3, 4, 5] - nuova lista ordinata
```

Copiare una lista

Attenzione: se scrivi `copia = originale`, non stai copiando la lista, stai solo creando un secondo nome per la stessa lista! Usa `.copy()`:

```
originale = [1, 2, 3]
copia = originale.copy() # copia vera e propria
copia.append(4)

print(originale) # [1, 2, 3] - non cambiata
print(copia)     # [1, 2, 3, 4] - modificata
```

Liste di liste

Puoi mettere liste dentro altre liste, utile per rappresentare griglie o tabelle:


```
matrice = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(matrice[1][2]) # 6 - seconda riga, terza colonna
```

Set (Insiemi)

Raccolte di elementi unici — perfetti per eliminare i duplicati e fare operazioni sugli insiemi.

Cosa sono i set?

Un **set** (insieme) è una raccolta di elementi con due regole fondamentali:

1. **Nessun duplicato**: ogni elemento può comparire al massimo una volta
2. **Nessun ordine garantito**: gli elementi non hanno una posizione fissa

Pensa a un set come a un sacchetto di palline colorate: sai quali colori ci sono dentro, ma non puoi dire "dammi la terza pallina".

Creare un set

Si usano le **parentesi graffe** `{}`, come per i dizionari, ma senza le chiavi:

```
frutta = {"mela", "banana", "ciliegia"}  
numeri = {1, 2, 3, 4, 5}
```

⚠ `{}` da solo crea un **dizionario vuoto**, non un set vuoto! Per creare un set vuoto usa `set()`:

```
vuoto = set() # set vuoto ✓  
anche_vuoto = {} # dizionario vuoto, non un set! ✗
```

:::

I duplicati vengono rimossi automaticamente:

```
con_duplicati = {1, 2, 2, 3, 3, 3}
print(con_duplicati)  # {1, 2, 3} - solo valori unici
```

Aggiungere e rimuovere elementi

```
frutta = {"mela", "banana"}

frutta.add("ciliegia")    # aggiunge un elemento
frutta.remove("banana")  # rimuove (dà errore se non esiste)
frutta.discard("uva")     # rimuove senza errore se non esiste
```

Controllare se un elemento è presente

```
frutta = {"mela", "banana", "ciliegia"}
print("mela" in frutta)    # True
print("uva" in frutta)     # False
```

I set sono molto veloci per questo tipo di ricerca, anche più delle liste.

Operazioni sugli insiemi

Qui i set diventano particolarmente utili: puoi fare operazioni tipiche della matematica degli insiemi.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Unione — tutti gli elementi di A e di B:

```
print(A | B)    # {1, 2, 3, 4, 5, 6}
```

Intersezione — solo gli elementi presenti in entrambi:

```
print(A & B)    # {3, 4}
```

Differenza — gli elementi in A ma non in B:

```
print(A - B)    # {1, 2}
```

Il trucco più utile: eliminare i duplicati

Il modo più semplice per eliminare i duplicati da una lista è convertirla in set e poi riconvertirla in lista:

```
lista = [1, 2, 2, 3, 3, 4, 1]
lista_pulita = list(set(lista))
print(lista_pulita)  # [1, 2, 3, 4]
```

:::note L'ordine originale non viene preservato, perché i set non hanno un ordine fisso. :::

Limiti dei set

- Non puoi accedere agli elementi per posizione: `frutta[0]` causa errore
- Gli elementi devono essere immutabili: puoi mettere numeri, stringhe, tuple, ma non liste

Stringhe in Python

Come lavorare con il testo in Python.

Cosa sono le stringhe?

Una **stringa** è semplicemente del testo: un nome, un messaggio, una frase. In Python le stringhe si scrivono tra virgolette singole `'...'` o doppie `"..."` :

```
nome = "Alice"  
saluto = 'Ciao, mondo!'
```

Usa virgolette doppie se dentro il testo c'è un apostrofo, e singole se c'è già una virgoletta doppia:

```
con_apice = "It's a great day"      # apostrofo dentro → virgolette doppie  
fuori  
con_virgolette = 'Disse "ciao!"'    # virgolette doppie dentro → singole  
fuori
```

Per testo su più righe, usa le virgolette triple:

```
poesia = """Questa stringa  
si estende su  
più righe."""
```

Accedere ai singoli caratteri

Come nelle liste, ogni carattere ha una posizione (indice) che parte da `0` :

```
testo = "Python"  
print(testo[0])  # P - primo carattere  
print(testo[1])  # y  
print(testo[-1]) # n - ultimo carattere  
print(testo[-2]) # o - penultimo
```

Estrarre una parte di stringa (slicing)

Puoi estrarre una sottostringa indicando da dove a dove:

```
testo = "Python"
print(testo[0:3])    # Pyt   - dalla posizione 0 alla 2 (3 esclusa)
print(testo[2:])     # thon  - dalla posizione 2 alla fine
print(testo[:4])     # Pyth  - dall'inizio alla posizione 3
print(testo[::-1])   # nohtyP - la stringa al contrario
```

Lunghezza di una stringa

```
testo = "Python"
print(len(testo))    # 6
```

Mettere stringhe insieme

Puoi unire due stringhe con `+` e ripetere una stringa con `*`:

```
a = "Ciao"
b = " mondo"
print(a + b)    # Ciao mondo
print(a * 3)    # CiaoCiaoCiao
```

Le f-string: il modo migliore per inserire variabili

Invece di unire le stringhe con `+` (che è scomodo), usa le **f-string**: scrivi una `f` prima delle virgolette e metti le variabili tra parentesi graffe `{}`:

```
nome = "Alice"
eta = 16
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Mi chiamo Alice e ho 16 anni.

a = 5
b = 3
print(f"La somma di {a} e {b} è {a + b}.")
# La somma di 5 e 3 è 8.
```

Le f-string sono la forma moderna e più leggibile. Usale sempre quando devi mettere variabili nel testo.

Metodi utili delle stringhe

Le stringhe hanno molti strumenti integrati. Ecco i più usati, divisi per categoria.

Cambiare maiuscole e rimuovere spazi

```
testo = "  Ciao, Mondo!  "

print(testo.upper())    # "  CIAO, MONDO!  " - tutto maiuscolo
print(testo.lower())    # "  ciao, mondo!  " - tutto minuscolo
print(testo.strip())    # "Ciao, Mondo!"    - rimuove spazi iniziali e finali
```

Cercare, sostituire e dividere

```
testo = "Ciao, Mondo!"

print(testo.replace("Mondo", "Python")) # "Ciao, Python!"

frutta = "mela,banana,ciliegia"
print(frutta.split(",")) # ['mela', 'banana', 'ciliegia'] - divide in lista
print(",".join(["mela", "banana", "ciliegia"])) # "mela,banana,ciliegia"
- unisce
```

Controllare il contenuto

```
testo = "ciao"

print(testo.capitalize()) # "Ciao" - prima lettera maiuscola
print(testo.startswith("ci")) # True - inizia con "ci"?
print(testo.endswith("ao")) # True - finisce con "ao"?
print(testo.find("ia")) # 1 - dove inizia "ia"? (-1 se non trovato)
print(testo.count("a")) # 1 - quante volte compare "a"?
```

Caratteri speciali

Alcune combinazioni di caratteri con `\` hanno un significato speciale:

Sequenza	Significato	Esempio
<code>\n</code>	Nuova riga	<code>"Prima riga\nSeconda riga"</code>
<code>\t</code>	Tabulazione (spazi)	<code>"Nome:\tAlice"</code>
<code>\\</code>	Un singolo backslash	<code>"C:\\Users\\Alice"</code>
<code>\"</code>	Virgolette doppie dentro la stringa	<code>"Disse \"ciao!\""</code>

```
print("Prima riga\nSeconda riga")
# Prima riga
# Seconda riga

print("Nome:\tAlice")
# Nome: Alice
```

Le stringhe non si modificano

Attenzione: le stringhe sono **immutabili**. Non puoi cambiare un singolo carattere. Se hai bisogno di una stringa modificata, devi crearne una nuova:

```
testo = "ciao"
# testo[0] = "C" - ERRORE! Non si può modificare

# Crea invece una nuova stringa
testo = "C" + testo[1:]
print(testo) # Ciao
```

Tuple

Sequenze di elementi che non cambiano mai.

Cosa sono le tuple?

Una **tupla** è simile a una lista: contiene elementi in ordine. La differenza fondamentale è che le tuple sono **immutabili**: una volta create, non si possono modificare. Non puoi aggiungere, togliere o cambiare gli elementi.

Pensa alle tuple come a dati "fissi": le coordinate GPS di un posto, i giorni della settimana, i colori del semaforo. Sono informazioni che non devono cambiare.

Creare una tupla

Le tuple si scrivono con le **parentesi tonde** `()` :


```
coordinate = (10, 20)
colori = ("rosso", "verde", "blu")
mista = (1, "ciao", True, 3.14)
vuota = ()
```

:::caution Una tupla con un solo elemento richiede una virgola finale, altrimenti Python la interpreta come una normale espressione tra parentesi:

```
non_e_tupla = (42)      # è solo il numero 42 tra parentesi!
e_una_tupla = (42,)     # la virgola la rende una tupla
```

:::

Accedere agli elementi

Funziona esattamente come per le liste, con gli indici:

```
colori = ("rosso", "verde", "blu")
print(colori[0])    # rosso
print(colori[-1])   # blu      - l'ultimo elemento
print(colori[1:3])  # ('verde', 'blu') - slicing
```

Non si possono modificare

Se provi a cambiare un elemento, Python dà errore:

```
colori = ("rosso", "verde", "blu")
colori[0] = "giallo" # TypeError: 'tuple' object does not support item
assignment
```

Questo è il comportamento voluto: garantisce che i dati rimangano intatti.

Operazioni sulle tuple

Anche se non si modificano, puoi comunque combinarle e controllarle:

```
a = (1, 2, 3)
b = (4, 5, 6)

print(a + b)    # (1, 2, 3, 4, 5, 6) – crea una nuova tupla unendo le due
print(a * 2)    # (1, 2, 3, 1, 2, 3) – ripete
print(3 in a)   # True – controlla se 3 è presente
print(len(a))  # 3 – numero di elementi
```

Spacchettare una tupla (unpacking)

Puoi assegnare gli elementi di una tupla a più variabili in una sola riga. Si chiama **unpacking** (spacchettamento):

```
coordinate = (10, 20)
x, y = coordinate
print(x)    # 10
print(y)    # 20

nome, eta, citta = ("Alice", 16, "Roma")
print(nome)  # Alice
print(eta)   # 16
print(citta) # Roma
```

Funziona anche con le liste. L'importante è che il numero di variabili corrisponda al numero di elementi.

Convertire tra tuple e liste

Se hai una tupla e hai bisogno di modificarla, convertila in lista, modificala e riconvertila:

```
t = (1, 2, 3)
l = list(t)      # converte in lista
l.append(4)      # modifica la lista
t = tuple(l)     # riconverte in tupla

print(t) # (1, 2, 3, 4)
```

Quando usare una tupla invece di una lista?

Usa una **tupla** quando:

- I dati non devono mai cambiare (coordinate, giorni della settimana, codici fissi)
- Vuoi comunicare a chi legge il codice che quei dati sono "di sola lettura"
- Hai bisogno di usarla come chiave di un dizionario (le liste non si possono usare come chiavi)

Usa una **lista** per tutto il resto, quando i dati possono cambiare.